

```

%-----
--
%                               Implementação do Metodo Simplex
%-----
--
% Variaveis principais:
%-----
--
% n = n° de variaveis originais
% m = n° de restrições funcionais
% A = matriz dos coeficientes técnicos
% b = vector dos termos independentes das restrições
% c = vector dos coeficientes das variaveis na FO
% x = vector com os indices de todas as variaveis
% xB = vector com os indices das variaveis básicas
% cB = vector com os coeficientes das variaveis básicas na FO
% Zjcj = vector com os valores da linha Zj-cj
% Z = valor da FO
% SBA = vector com os valores da SBA em cada iteração
%-----
--

%limpa janela de comandos
clc
disp('-----')
disp('          Resolução de um problema pelo método Simplex          ')
disp('-----')
disp(' Assume-se que:                                                    ')
disp(' -> Função objectivo está na forma de maximização                    ')
disp(' -> Todas as restrições são de "<="                                    ')
disp(' -> Todas as variáveis são >=0                                         ')
disp('-----')
% Le dados do problema
[n,m,c,A,b]=Le_dados;

%limpa novamente janela de comandos
clc

I=eye(m);           % Matriz identidade (mxm)
A=[A I];            % Matriz A do problema na forma aumentada
cs=zeros(1,m);      % Coeficientes das variaveis slack na FO
c=[c cs];           % Coeficientes de todas as variaveis na FO

xo=1:n;             % Indices das variaveis originais
xs=n+1:n+m;         % Indices das variaveis slack
x=[xo xs];          % Indices de todas as variaveis

xB=xs';             % xB e um vector coluna com os indices das VBs
cB=cs';             % cB e um vector coluna com os coeficientes das VBs na FO

SBA=[zeros(n,1);b]; % Inicialização da 1ª SBA (vector coluna)

Zjcj=zeros(1,n+m);  % Inicialização do vector Zjcj a zeros

termina=0;          % Controla a execução do ciclo
iteracao=1;         % Contabiliza o n° de iterações
while ~termina
    % Calcula a linha Zj-cj (Vector ZjCj)

```

```

for j=1:n+m
    Zjcj(j)=cB'*A(:,j)-c(j);
end

% Calcula o valor de Z
Z=cB'*b;

% Testa se solucao e optima => Procura o valor minimo da linha Zj-cj
% Função 'min' - devolve o valor mínimo e a posição que ocupa no
vector
[valor_min,coluna_pivot]=min(Zjcj);

if valor_min>=0      % Solução é optima => termina ciclo
    termina=1;
    % Apresenta quadro óptimo do Simplex
    Apresenta_quadro_Simplex(n,m,c,xB,cB,A,b,Zjcj,Z,iteracao,0,0,0)
    % Apresenta a solucao optima e o valor de Z optimo
    Apresenta_resultados_finais(n,m,SBA,Z);

else                % Senao, realiza nova iteracao
    % Calcula os quocientes entre os valores de b e da coluna pivot
    q=realmax*ones(m,1); % Inicializa elementos de q com o maior
numero real possivel
    for i=1:m
        if A(i,coluna_pivot)>0 % So sao possiveis quocientes sobre
valores >0
            q(i)=b(i)/A(i,coluna_pivot);
        end
    end
    % Determina linha pivot
    [valor_min,linha_pivot]=min(q);

    % Variaveis que vão entrar e sair da base
    % -> Variavel que vai entrar: x(coluna_pivot)
    % -> Variavel que vai sair: xB(linha_pivot))

    % Apresenta quadro Simplex

Apresenta_quadro_Simplex(n,m,c,xB,cB,A,b,Zjcj,Z,iteracao,1,x(coluna_pivot
),xB(linha_pivot))

    pause % Espera que o utilizador carregue numa tecla

    % Actualiza base
    xB(linha_pivot)=x(coluna_pivot);
    cB(linha_pivot)=c(coluna_pivot);

    % Reduz elemento pivot à unidade dividindo toda a linha pivot e
% respectivo valor de b pelo inverso do valor actual do pivot
    k=1/A(linha_pivot,coluna_pivot);
    A(linha_pivot,:)=k*A(linha_pivot,:);
    b(linha_pivot)=k*b(linha_pivot);

    % Elimina restantes elementos da coluna pivot usando a seguinte
% operação: (i)' = (i) -k*(linha pivot)'
    % Notar que 'k' é o valor que se pretende eliminar
    for i=1:m
        % Exclui a linha pivot

```

```

        if(i~=linha_pivot)
            k=A(i,coluna_pivot);
            A(i,:)=A(i,:)-k*A(linha_pivot,:);
            b(i)=b(i)-k*b(linha_pivot);
        end
    end

% Actualiza os valores da SBA
for j=1:n+m
    % Verifica se a variavel de indice j esta na base
    indexVB=find(xB==j);
    % A função 'find' verifica se algum elemento de xB é igual a
j
    % e se for devolve a sua posição. Senão, indexVB fica uma
matriz
    % vazia
    if ~isempty(indexVB)      % Índice j faz parte da base
        SBA(j)=b(indexVB);    % Vai buscar valor de xj à coluna b
    else
        SBA(j)=0;             % Se não faz parte da base, xj é
zero
    end
end
end
iteracao=iteracao+1;
end

```